Árbol tipo C4.5 y bosque aleatorio sobre Breast Cancer Wisconsin

Celeste Martin Rodich[1], Matías Emmanuel Machado Benítez[1], Camila Aylen Zeniquel Martinelli[1], Nadine Andrea Peralta Ruiz[1], and Mauricio Nicolás Castro[1]

Universidad Tecnológica Nacional, Facultad Regional Resistencia, Argentina

Resumen Se compara un árbol de decisión escrito por el equipo con un bosque aleatorio de scikit-learn. El archivo de la cátedra no estaba disponible, así que los números son del respaldo Breast Cancer Wisconsin (diagnóstico): 569 casos, 30 atributos continuos, sin faltantes, 357 benign y 212 malignant. La métrica que decide es el F1 macro. En 25 particiones estratificadas el bosque llega a 0,9530 ± 0,0251 y el árbol a 0,9221 ± 0,0236. La poda pesimista, con factor de confianza 0,25, baja el árbol de 47,64 a 30,68 nodos y la brecha de accuracy de 0,0731 a 0,0598. No se corrieron barridos de hiperparámetros. El árbol no es un C4.5 completo: un atributo continuo se corta una sola vez, la poda solo reemplaza un subárbol por una hoja y el factor t es propio del equipo.

Keywords: árbol de decisión · C4.5 · random forest · gain ratio · poda pesimista

1. Introducción

El trabajo pide comparar dos clasificadores sobre el mismo protocolo. El árbol de decisión lo escribe el equipo. Para el bosque, el enunciado habilita un modelo de biblioteca: "para el algoritmo RF se podrá utilizar un modelo de la libraría Scikit-learn". Se conserva la grafía *libraría* porque así figura en la consigna.

La pregunta es concreta: con los mismos datos y la misma semilla, cuál modelo clasifica mejor, de qué tamaño queda y cuánto tarda en ajustarse. La respuesta de este informe vale para el respaldo Breast Cancer Wisconsin [5], que es el dataset con el que se midió. El CSV de la cátedra no estaba en el repositorio.

La numeración de figuras y tablas es la del notebook. Entran la Tabla 2, la Tabla 5 y las Figuras 4, 7, 9 y 10.

2. Algoritmos

2.1. Árbol

El árbol elige el corte con gain ratio: la ganancia de información dividida por la información de la partición, para no premiar a un atributo solo porque abre

muchas ramas [2, p. 67]. La entropía se calcula sobre las etiquetas de la clase. Dos filas de la misma clase aportan entropía 0.

Un atributo categórico abre una rama por valor. Un continuo se parte en dos, con rama izquierda $\leq$ umbral y rama derecha $>$ umbral. Los candidatos son los puntos medios entre valores ordenados donde cambia la clase. En este respaldo, esos umbrales coincidieron con los de revisar todos los puntos medios: 3 527 candidatos en lugar de 11 002, y los 30 umbrales elegidos fueron los mismos. Después de usar un atributo, el árbol lo saca de la lista. Un continuo no se vuelve a cortar más abajo en la misma rama.

Si falta el valor del atributo que parte, el caso se reparte entre las ramas conocidas en proporción a la masa de cada rama. En este respaldo no hubo faltantes, así que esa rama del código no entra en los números de la Sección 5.

Cada nodo guarda $t = n_{\text{conocidos}}/n_{\text{total}}$, la proporción de peso para la que se conoce el atributo elegido. En una hoja, $t = 1$. Ese factor lo agrega el equipo para la poda con masa fraccionaria. No forma parte de la rutina AddErrs.

La poda es pesimista y corre después de crecer el árbol. En cada nodo se compara el error estimado de dejar el subárbol con el de reemplazarlo por una hoja. Si la hoja no es peor, el nodo se convierte en hoja. El error estimado es $e+u(e, N, CF)$, con $CF = 0{,}25$. La función u sigue la rutina AddErrs de Quinlan [4, pp. 278–279]: interpola con CF y, para 0,25, obtiene $z = 0{,}6925$. El código no usa la convención $CF/2$ de la nota al pie de la sección 4.2 de ese libro.

2.2. Bosque

El bosque es el de Breiman [1]. Se toma de scikit-learn [3], en la clase `RandomForestClassifier`, y queda envuelto en `RandomForestWrap`. La corrida publicada usa 100 árboles y la semilla 42. Cada árbol ve la raíz cuadrada de los atributos, con remuestreo y hoja mínima de 1. No hay tope de profundidad.

Scikit-learn no recibe columnas categóricas en crudo. El wrapper imputa las numéricas con la mediana y las categóricas con la moda, y aplica one-hot a estas últimas. Ese paso ocurre dentro de `fit` y de `predict`. Quien compara los modelos les pasa las mismas columnas.

3. Solución y problemas

Los dos modelos exponen `fit(X, y)`, `predict`, `predict_proba` y `score`. `X` es un `DataFrame`. La primera versión del wrapper entrenaba con la firma del árbol mínimo: un `DataFrame` y el nombre de la clase en el mismo argumento. Con esa firma y la de `DecisionTreeC45` no había una sola función de evaluación. El wrapper pasó a `fit(X, y)`.

El árbol que se mide no es un C4.5 completo. Tres diferencias están en el código y hay que leerlas como tales:

- Un atributo, también si es continuo, se usa una sola vez. C4.5 puede volver a cortar un continuo más abajo, con otro umbral.

- La poda reemplaza un subárbol por una hoja. No hace el otro movimiento de Quinlan, que cambia el subárbol por su rama más usada.
- El factor t es una decisión del equipo para ponderar la masa conocida. No hay que atribuirlo a AddErrs.

El criterio por defecto no impone un gain ratio mínimo (`min_gain_ratio` vale 0) y `min_samples_leaf` vale 1. La poda viene apagada: `pruning=False`. La Tabla 5 la enciende a propósito, con $CF = 0{,}25$.

4. Metodología

La semilla global es 42. El validador compartido es una validación estratificada repetida, `RepeatedStratifiedKFold`, con 5 pliegues y 5 repeticiones: 25 scores. Cada pliegue mantiene la proporción de clases. Sobre este respaldo, los 25 pliegues de prueba tienen las dos clases. `evaluar_cv` usa ese objeto y rechaza otro splitter. Pide también el score de entrenamiento. La brecha es ese score menos el de prueba.

La métrica de comparación es el F1 macro: cada clase pesa igual. Accuracy y las medias ponderadas se publican y no eligen al ganador. Con dos clases, la positiva para la matriz y para el ROC es la segunda de `classes_`. En este archivo es `malignant`.

El corte final es un holdout estratificado 80/20, una sola vez, con la misma semilla. En el respaldo son 455 filas de entrenamiento y 114 de prueba. El resultado queda en `figuras/holdout_final.json`. No se usa validación cruzada anidada.

Los barridos de hiperparámetros (experimentos E2 a E11) no se corrieron. Los modelos de las tablas usan los valores por defecto del párrafo anterior. Cuando esos barridos se hagan, tienen que ver solo el entrenamiento: el holdout no entra a elegir un parámetro.

La Tabla 2, la Tabla 5 y las Figuras 7 y 9 se calcularon sobre todas las filas, antes de fijar el holdout. Describen el comportamiento de los modelos. El holdout de la Sección 5 es la evaluación final. Una corrida anterior del árbol con `max_depth=2` no se mezcla con estas tablas: el árbol de la Tabla 2 no tiene tope de profundidad.

5. Resultados

En F1 macro el bosque queda arriba: $0{,}9530 \pm 0{,}0251$ contra $0{,}9221 \pm 0{,}0236$ del árbol. La accuracy de prueba es $0{,}9564 \pm 0{,}0232$ y $0{,}9269 \pm 0{,}0219$. El ROC-AUC es $0{,}9893 \pm 0{,}0092$ y $0{,}9246 \pm 0{,}0256$. El árbol tiene unos 48 nodos; el bosque, sumando sus 100 árboles, unos 3 513. Ajustar el árbol lleva $26{,}04 \pm 4{,}78$ segundos por pliegue y el bosque $0{,}191 \pm 0{,}043$. Predecir es barato en los dos. La Tabla 2 trae media y desvío en columnas separadas. La Figura 4 muestra las métricas de prueba con el desvío como barra de error.

Tabla 2. DT vs RF: media y desvío en test, tiempos (s) y tamaño.

metrica	DecisionTreeC45 media	DecisionTreeC45 desvio	RandomForestWrap media	RandomForestWrap desvio
accuracy	0.9269	0.0219	0.9564	0.0232
precision_macro	0.9212	0.0222	0.9573	0.0232
precision_weighted	0.9285	0.0218	0.9581	0.0220
recall_macro	0.9246	0.0256	0.9511	0.0277
recall_weighted	0.9269	0.0219	0.9564	0.0232
f1_macro	0.9221	0.0236	0.9530	0.0251
f1_weighted	0.9270	0.0220	0.9562	0.0234
roc_auc	0.9246	0.0256	0.9893	0.0092
tiempo_fit	26.0373	4.7815	0.1912	0.0430
tiempo_predict	0.0254	0.0108	0.0386	0.0252
nodos	47.6400	5.9643	3512.9600	169.9550
hojas	24.3200	2.9822	1806.4800	84.9775
altura	13.2000	1.9365	10.2000	0.9574

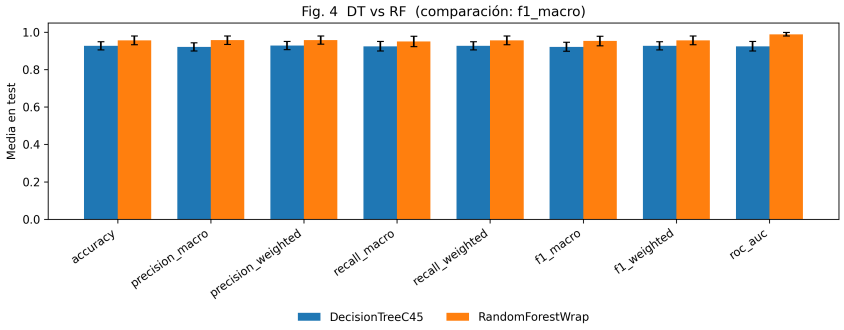


Figura 4. Métricas de prueba, media y desvío, en las 25 particiones.

Sin poda el árbol acierta todo el entrenamiento (accuracy 1) y la brecha es 0,0731. Con poda pesimista baja a 30,68 nodos y 15,84 hojas, la accuracy de entrenamiento a 0,9902 y la de prueba sube a 0,9304. La brecha queda en 0,0598. El bosque, sin poda que regularice cada árbol por separado, tiene brecha 0,0436. La baja de 0,0731 a 0,0598 achica el sobreajuste y no llega a una brecha de unos 3 puntos. La Tabla 5 y la Figura 7 usan las mismas 25 particiones.

En el holdout, con la positiva `malignant`, el árbol tiene accuracy 0,9298, F1 macro 0,9246 y ROC-AUC 0,9246. El bosque tiene 0,9737, 0,9713 y 0,9929. La matriz del árbol es 68, 4, 4 y 38 (TN, FP, FN, TP). La del bosque es 72, 0, 3 y 39. En las 114 filas de prueba hay 72 benign y 42 malignant. El árbol se equivoca en 4 de 72 benign (5,6 %) y en 4 de 42 malignant (9,5 %). El bosque no marca ningún benign como malignant y deja pasar 3 de 42 malignant (7,1 %). El árbol reparte el error. El bosque lo concentra en la clase chica. Las Figuras 9 y 10 salen de este corte. La matriz de la figura está normalizada por fila.

Tabla 5. DT con poda pesimista y sin poda: media y desvío en 25 particiones.

metrica	sin_poda media	sin_poda desvio	con_poda media	con_poda desvio
nodos	47.6400	5.9643	30.6800	5.0885
hojas	24.3200	2.9822	15.8400	2.5443
altura	13.2000	1.9365	10.6400	2.2338
accuracy_train	1.0000	0.0000	0.9902	0.0058
accuracy_test	0.9269	0.0219	0.9304	0.0204
brecha	0.0731	0.0219	0.0598	0.0215

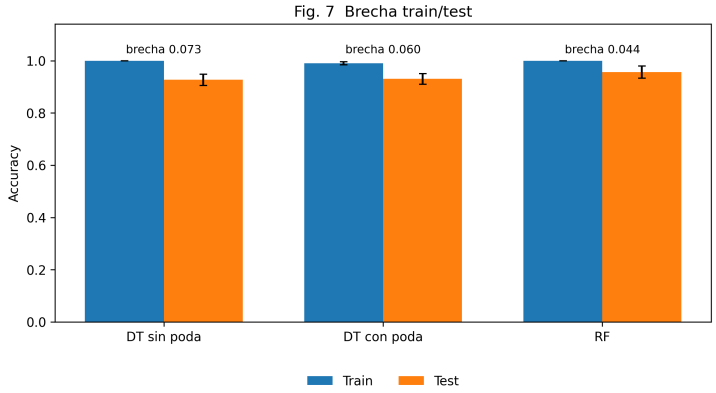


Figura 7. Accuracy de entrenamiento y de prueba. La brecha es la diferencia de las medias.

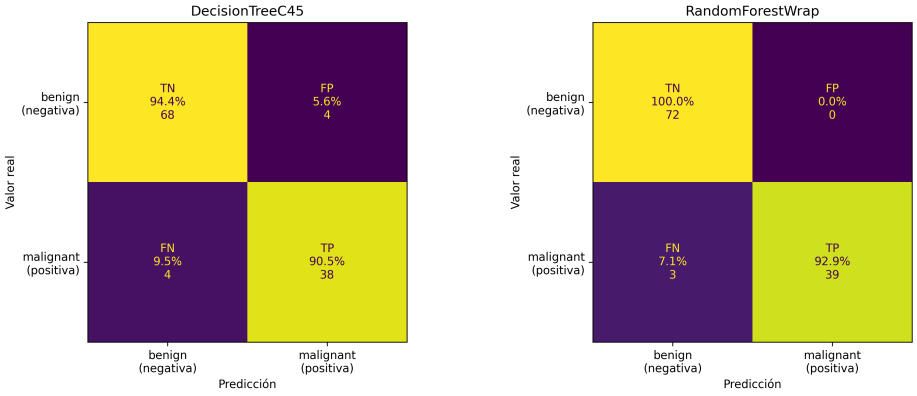


Figura 9. Matrices de confusión del holdout, normalizadas por fila.

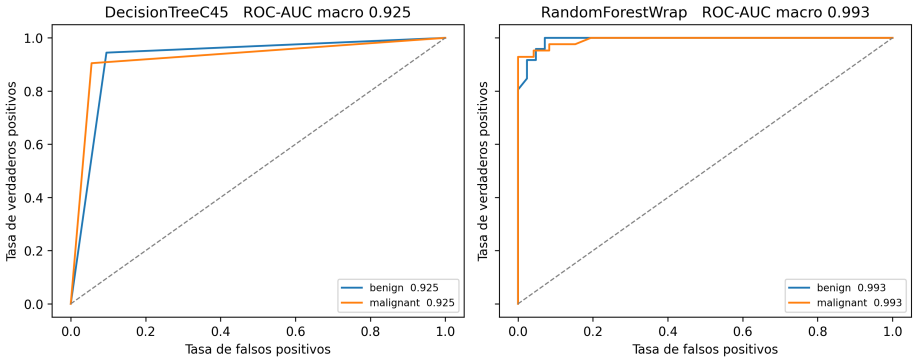


Figura 10. Curva ROC por clase en el holdout. El título de cada panel es el ROC-AUC macro.

6. Discusión

Estos números describen un solo dataset de respaldo. No dicen qué va a pasar con el CSV de la cátedra, que puede traer categóricas, faltantes y otra proporción de clases. El manejo fraccional de faltantes y el factor t quedaron escritos y no tuvieron casos que partir en esta medición.

Tampoco hubo barridos. Profundidad, cantidad de árboles, cantidad de atributos por corte y tamaño mínimo de hoja están en el valor por defecto. Una elección hecha sobre el entrenamiento podría mover la diferencia de F1. Hasta que eso se corra, la comparación es la de las implementaciones con sus valores por defecto.

La poda se juzga por la media de las 25 particiones. En un holdout de la misma semilla, el mismo árbol pasó de 53 a 31 nodos y la accuracy de prueba bajó de 0,9298 a 0,9211. La decisión de podar mira el entrenamiento. Un corte solo no garantiza que el holdout suba, aunque la media de las 25 particiones sí suba de 0,9269 a 0,9304.

El tiempo de ajuste no se lee como una propiedad del algoritmo en abstracto. El árbol está escrito en Python y recorre candidatos en el intérprete. El bosque llama a una implementación compilada. Los 26 segundos frente a 0,19 segundos son el costo medido en esta corrida.

7. Conclusiones

En Breast Cancer Wisconsin, con semilla 42 y sin buscar hiperparámetros, el bosque de scikit-learn supera al árbol del equipo en F1 macro, en accuracy y en ROC-AUC, y se ajusta en una fracción del tiempo. El árbol queda dos órdenes de magnitud más chico.

La poda pesimista con $CF = 0{,}25$ reduce nodos, hojas, altura y la brecha entre entrenamiento y prueba, y sube un poco la accuracy media de prueba. Eso es un resultado de 25 particiones. En un holdout, la accuracy de prueba puede bajar.

El árbol implementa gain ratio, umbrales continuos, faltantes fraccionarios y la estimación AddErrs. Sigue en deuda con un C4.5 completo en tres puntos ya dichos: un solo corte por atributo continuo, poda sin *subtree raising* y un factor t propio. Cerrar la comparación pide el dataset de la cátedra y los barridos que todavía no se corrieron.

Referencias

1. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
2. Liu, B.: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data. Springer, 2 edn. (2011)
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)

4. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann, San Mateo, CA (1993)
5. Wolberg, W.H., Street, W.N., Mangasarian, O.L.: Breast cancer Wisconsin (diagnostic). UCI Machine Learning Repository (1995)